

1. Indicate whether the following statements are true or false. Justify your answers.

(a) In the dining-philosopher problem with five philosophers, if we allow at most four of them to be hungry simultaneously, deadlock may still occur.

(b) It is impossible to have a deadlock involving only one single process.

(c) If a resource allocation graph contains a cycle, then a deadlock has occurred.

1a) In the dining-philosopher problem with five philosophers, if we allow at most four of them to be hungry simultaneously, deadlock may still occur.

→ False.

*Justification:* Deadlock does not occur, since at least one philosopher can get both chopsticks.

1b) It is impossible to have a deadlock involving only one single process.

→ True.

*Justification:* The hold-and-wait condition can never be satisfied.

1c) If a resource allocation graph contains a cycle, then a deadlock has occurred.

→ False.

*Justification:* It depends on the number of instances per resource type.

2. A resource-allocation state is given below. Assume Available = 2

|           | Allocation | Max | Need |
|-----------|------------|-----|------|
| PROCESS 1 | 1          | 6   | 5    |
| PROCESS 2 | 1          | 5   | 4    |
| PROCESS 3 | 2          | 4   | 2    |
| PROCESS 4 | 4          | 7   | 3    |

- (a) If PROCESS 4 requests for one unit of the resource next, does this lead to a safe state or an unsafe one?
- (b) If PROCESS 3 requests for one unit of the resource next, does this lead to a safe state or an unsafe one?

# If PROCESS 4 requests for one unit...

|           | Allocation | Max | Need |
|-----------|------------|-----|------|
| PROCESS 1 | 1          | 6   | 5    |
| PROCESS 2 | 1          | 5   | 4    |
| PROCESS 3 | 2          | 4   | 2    |
| PROCESS 4 | 4          | 7   | 3    |

Available = 2



|           | Allocation | Max | Need |
|-----------|------------|-----|------|
| PROCESS 1 | 1          | 6   | 5    |
| PROCESS 2 | 1          | 5   | 4    |
| PROCESS 3 | 2          | 4   | 2    |
| PROCESS 4 | 5          | 7   | 2    |

Available = 1

→ unsafe, since  $Need_i > Available$  for all  $i$ .

# If PROCESS 3 requests for one unit...

|           | Allocation | Max | Need |
|-----------|------------|-----|------|
| PROCESS 1 | 1          | 6   | 5    |
| PROCESS 2 | 1          | 5   | 4    |
| PROCESS 3 | 2          | 4   | 2    |
| PROCESS 4 | 4          | 7   | 3    |

Available = 2



|           | Allocation | Max | Need |
|-----------|------------|-----|------|
| PROCESS 1 | 1          | 6   | 5    |
| PROCESS 2 | 1          | 5   | 4    |
| PROCESS 3 | 3          | 4   | 1    |
| PROCESS 4 | 4          | 7   | 3    |

Available = 1

→ Safe, since sequence <PROCESS 3, PROCESS 2, PROCESS 1, PROCESS 4> satisfies safety requirement.

3. Consider the following snapshot of a system's state, with five processes (P0, P1, P2, P3 and P4) and four resource types (A, B, C and D). The current **Allocation**, **Need** and **Available** matrices are as shown in the table below.

|                |                   |                                        |
|----------------|-------------------|----------------------------------------|
|                |                   | <b>Available</b><br>A B C D<br>0 x y 1 |
| <b>Process</b> | <b>Allocation</b> | <b>Need</b>                            |
|                | A B C D           | A B C D                                |
| P0             | 2 2 0 0           | 1 4 0 0                                |
| P1             | 3 1 0 0           | 0 1 0 0                                |
| P2             | 1 1 0 0           | 1 0 0 0                                |
| P3             | 0 0 1 1           | 0 0 1 1                                |
| P4             | 0 0 1 1           | 0 0 3 2                                |

Suppose process P3 generates the resource request (A=0 B=0 C=0 D=1) next. Compute the minimum values for **x** and **y** so that this request from P3 can be granted and the resulting system state is safe. Justify your answer and show the algorithm steps for generating the safe sequence.



**The minimum values for x and y are 2 and 2, respectively. The resulting safe sequence is <P1,P2,P3,P0,P4>.**

Note, A and B are only used by P0-P2 and C and D are only used by P3-P4. Although P1 and P2 can be processed with x=1, we need x=2 to complete P0. Although P3 can be processed with y=1, we need y=2 to complete P4.

Initially, Available matrix is A=0 B=(x=2) C=(y=2) D=1. Upon, satisfying P3's request for A=0 B=0 C=0 D=1, Available becomes 0 2 2 0 and P3's Need becomes 0 0 1 0 and Allocation becomes 0 0 1 2.

Running the safety algorithm, Available matrix evolves as follows.

| <u>Available</u> | <u>Need</u> | <u>Allocation</u> | <u>Process completion</u> |
|------------------|-------------|-------------------|---------------------------|
|                  | A B C D     | A B C D           |                           |
| 0 2 2 0          | 0 1 0 0     | 3 1 0 0           | P1                        |
| 3 3 2 0          | 1 0 0 0     | 1 1 0 0           | P2                        |
| 4 4 2 0          | 0 0 1 0     | 0 0 1 2           | P3                        |
| 4 4 3 2          | 1 4 0 0     | 2 2 0 0           | P0                        |
| 6 6 3 2          | 0 0 3 2     | 0 0 1 1           | P4                        |
| 6 6 4 3          |             |                   |                           |